

Enhancing the MADDPG Algorithm via Pre-Trained Action Inference in Competitive Multi-Agent Environments

Shaash Sivakumar, Marc Walden, Jason Liu, Ryan Liu, Hamza Khan, Henry Cronin

UCLA Mathematics Department

May 2025

Abstract

We investigate multi-agent deep reinforcement learning in competitive environments and propose an enhancement to the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm through a novel *Pre-Trained Action Inference* (PTAI) mechanism. PTAI pre-trains a dedicated neural network offline—using only randomly generated interaction data—to predict each agent’s previous joint action vector from temporally stacked observations. These inferred actions are then concatenated to each agent’s local observation before it enters the actor network, providing a richer behavioural context for policy learning without introducing any new online gradient interactions. We evaluate both baseline MADDPG and MADDPG+PTAI on four competitive Mixed Physical Environments (MPE) from the PettingZoo library across 3–4 independent random seeds each. Across these runs, PTAI achieves higher mean final-policy performance in all four environments, with observed relative improvements of +104% in Physical Deception (`simple_adversary_v3`), +83% in Cooperative Push (`simple_push_v3`), +157% in Keep-Away (`simple_world_comm_v3`), and +437% in Predator-Prey (`simple_tag_v3`). Given the small number of independent training runs, these results are best interpreted as promising empirical evidence motivating more systematic statistical evaluation rather than as precise estimates of population-level effect sizes.

Contents

1	Introduction	3
2	Background	4
2.1	Reinforcement Learning Framework	4
2.2	Deep Q-Learning and Multi-Agent Extensions	4
2.3	Policy-Gradient Methods and MADDPG	4

3	Method: Pre-Trained Action Inference (PTAI)	5
3.1	Motivation	5
3.2	AI_Net Architecture	5
3.3	Pre-Training Procedure	6
3.4	Integration with MADDPG	6
4	Environments	6
5	Experiments	8
5.1	Training Setup	8
5.2	Evaluation Protocol	8
6	Results	9
6.1	Quantitative Summary	9
6.2	Training Convergence	10
7	Discussion	11
7.1	Interpretation of the Observed Gains	11
7.2	A Statistical View of the Simulation Study	12
7.3	Information-Theoretic Perspective	12
7.4	Limitations and Planned Evaluation	13
8	Conclusion	14
A	Algorithms	16

1 Introduction

Multi-agent reinforcement learning (MARL) addresses settings in which multiple autonomous agents must simultaneously learn effective policies while interacting within a shared environment [1]. Such settings arise naturally in robotics, autonomous vehicles, strategic game playing, and network resource allocation, among other domains. A defining challenge in MARL is *non-stationarity*: from any single agent’s perspective, the environment appears to change over time because the policies of all other agents are simultaneously evolving. This violates the Markov property that underpins most single-agent RL convergence results, and can cause independent learners to oscillate or diverge [11].

Actor-critic algorithms partially address non-stationarity by combining a low-variance critic baseline with a policy-gradient actor update [7]. The Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm [5] goes further by adopting a *centralised training, decentralised execution* (CTDE) paradigm: each agent maintains a centralised critic that observes the full joint observation-action tuple during training, while each actor relies only on the agent’s local observation at execution time. This formulation substantially stabilises learning because the centralised critic can directly account for the actions of all agents.

Even under CTDE, the decentralised actor is conditioned only on the agent’s current local observation at execution time. In competitive settings, two similar current observations may arise from different recent interaction histories and can call for different responses. Previous joint actions therefore provide a natural source of temporal context: they can help distinguish how the current configuration was reached and reveal short-term behaviour of teammates or opponents. Standard MADDPG does not provide this action history explicitly to the actor.

We address this gap with *Pre-Trained Action Inference* (PTAI). A dedicated AI_Net module is trained offline—before any MADDPG gradient updates—on random interaction data to predict the previous joint action vector from a pair of consecutive local observations. Once pre-trained, the AI_Net is frozen and used at every decision step to supply each agent’s actor with an estimate of the previous joint action, concatenated to its current observation. Because the AI_Net is stationary and requires no online updates, it does not introduce a second jointly optimised objective during MADDPG training. At the same time, it augments each actor with temporal context that is not supplied explicitly by standard MADDPG.

Contributions.

1. We describe a modular PTAI architecture that exploits the structural separability of MPE observations to infer previous joint actions efficiently via Directional Social and Self-Awareness modules.
2. We extend the evaluation of PTAI from a single environment (Predator-Prey) to four competitive MPE environments, with 3–4 independent seeds per condition.
3. We report quantitative, seed-averaged results with standard deviations, observing higher mean final-policy performance for PTAI than baseline MADDPG across all four tested environments.

For the initial implementation of standard MADDPG we adapted an open-source code-base [3], then rebuilt and extended it with the PTAI mechanism described here.

2 Background

2.1 Reinforcement Learning Framework

In a standard reinforcement learning (RL) setting, an agent interacts with an environment $\mathcal{E}$ over discrete time steps $t = 0, 1, 2, \dots$ [10]. At each step, the agent observes $o_t^i \in \mathcal{O}$, selects an action $a_t^i \in \mathcal{A}$, and receives a scalar reward $r_{t+1}^i \in \mathbb{R}$. The agent’s goal is to learn a policy $\pi_{\theta_i}(a^i | o^i)$, parameterised by θ_i , that maximises the expected discounted return:

$$J(\theta_i) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1}^i \right], \quad (1)$$

where $\gamma \in [0, 1)$ is the discount factor. The most common approach to optimising Equation (1) is General Policy Iteration (GPI), which alternates between policy evaluation (estimating the current value function) and policy improvement (refining the policy to increase expected value).

2.2 Deep Q-Learning and Multi-Agent Extensions

Mnih et al. [6] introduced the Deep Q-Network (DQN), which approximates the action-value function $Q(s, a)$ with a deep neural network and minimises the Bellman error

$$\mathcal{L}(\phi) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q_{\phi}(s', a') - Q_{\phi}(s, a) \right)^2 \right], \quad (2)$$

where $\mathcal{D}$ is an experience replay buffer. DQN excels in single-agent settings but becomes unstable in multi-agent environments because each agent’s policy updates effectively change the environment dynamics for all others.

Tan [11] applied independent Q-learning to multi-agent problems, but the resulting non-stationarity is well documented. Addressing this motivated the centralised critic approach of MADDPG.

2.3 Policy-Gradient Methods and MADDPG

Policy-gradient methods directly optimise $J(\theta)$ by gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$. The Policy Gradient theorem [9] gives:

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a | o) Q^{\pi}(o, a)]. \quad (3)$$

MADDPG [5] extends deterministic policy gradients [8] to the multi-agent setting by giving each agent i a decentralised actor $\mu_{\theta_i}(o_i)$ and a centralised critic $Q_{\phi_i}(x, a_{1:N})$, where x is the global joint observation and $a_{1:N}$ the joint action. Each critic is trained to minimise

$$\mathcal{L}(\phi_i) = \frac{1}{S} \sum_j (y^j - Q_{\phi_i}(x^j, a_1^j, \dots, a_N^j))^2, \quad y^j = r_i^j + \gamma Q_{\phi_i'}(x'^j, a_1'^j, \dots, a_N'^j), \quad (4)$$

and each actor is updated by the sampled deterministic policy gradient:

$$\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o_i^j) \nabla_{a_i} Q_{\phi_i}(x^j, a_1^j, \dots, a_i, \dots, a_N^j) \Big|_{a_i = \mu_i(o_i^j)}. \quad (5)$$

During execution each agent acts based only on its local observation o_i , preserving scalability and decentralisation. Target networks (soft-updated at rate τ) and a shared experience replay buffer ensure stable training. The full algorithm is given as Algorithm 1 in the appendix.

3 Method: Pre-Trained Action Inference (PTAI)

3.1 Motivation

In competitive MARL, the quality of an agent’s decision depends not only on its current observation but also on what other agents have just done. Two otherwise identical states can call for very different actions depending on whether, e.g., an opponent just moved aggressively or retreated. Standard MADDPG actors have no direct access to this temporal context: the actor sees only the current observation o_i^t , and the previous joint action $(a_1^{t-1}, \dots, a_N^{t-1})$ is not stored separately in the standard replay buffer.

PTAI provides this missing temporal signal by inferring the previous joint action from the change in observations—without requiring communication between agents or access to opponents’ policies.

3.2 AI_Net Architecture

Let N be the number of agents. Each agent i maintains its own instance of an *AI_Net* that maps

$$\hat{\mathbf{a}}_i = \text{AI_Net}(o_i^t, o_i^{t-1}), \quad (6)$$

where $o_i^t, o_i^{t-1} \in \mathbb{R}^{d_i}$ are the current and previous observations of agent i , and $\hat{\mathbf{a}}_i \in \mathbb{R}^{\sum_k |A_k|}$ is an estimate of the full previous joint action vector $\text{concat}(\text{onehot}(a_1^{t-1}), \dots, \text{onehot}(a_N^{t-1}))$.

The key structural insight is that many MPE observation vectors are *separable*: different contiguous segments correspond to features of different agent types. This allows AI_Net to decompose the prediction task into smaller specialised modules rather than one monolithic network (which would scale as $O(N^2)$ in parameters).

Directional Social Awareness Modules. For each ordered pair (observer type p , observed type q), a Social Awareness module takes as input the features in o_i^t and o_i^{t-1} that pertain to an agent of type q , together with the self/global features of observer i , and outputs a prediction of that agent’s previous action:

$$\hat{a}_{i,k} = \text{Social}_\theta(o_{i,k}, o_{i,g}, o_{i,k}^-, o_{i,g}^-, \Delta o_{i,k}, \Delta o_{i,g}), \quad (7)$$

where $o_{i,k}$ are the features in agent i ’s observation relating to agent k , $o_{i,g}$ are global/self features, superscript $-$ denotes the previous-step value, and $\Delta = o^t - o^{t-1}$ is the temporal difference.

Directional Self-Awareness Module. A Self-Awareness module predicts the agent’s own previous action from its self/global features and their temporal difference:

$$\hat{a}_{i,i} = \text{Self}_{\theta}(o_{i,i}, o_{i,g}, o_{i,i}^-, o_{i,g}^-, \Delta o_{i,i}, \Delta o_{i,g}). \quad (8)$$

The complete joint action estimate is $\hat{\mathbf{a}}_i = (\hat{a}_{i,1}, \dots, \hat{a}_{i,N})$, where each component is the output of the appropriate Social or Self module routed by AI_Net according to agent types.

3.3 Pre-Training Procedure

AI_Net is pre-trained offline on $P = 200$ episodes of random interaction (all agents sampling uniformly from their action spaces) prior to any MADDPG training. At each step, all awareness modules are updated by minimising the mean-squared error between predicted and true one-hot actions (Algorithm 2 in the appendix). A stochastic sampling gate randomly drops a fraction of training pairs to improve generalisation.

After pre-training, the weights of all modules are *frozen*. Because the AI_Net is stationary, it introduces no additional gradient optimisation interaction with the MADDPG training loop. It does, however, add a forward-pass inference cost when constructing the actor input; this overhead is separate from MADDPG’s gradient updates.

3.4 Integration with MADDPG

At each decision step $t \geq 2$, for each agent i , the frozen AI_Net is queried:

$$\hat{\mathbf{a}}_i^t = \text{AI_Net}(o_i^t, o_i^{t-1}). \quad (9)$$

The actor input is extended to:

$$\mu_{\theta_i}([o_i^t \parallel \hat{\mathbf{a}}_i^t]), \quad (10)$$

where $\parallel$ denotes concatenation. At the first step of each episode (no previous observation available), $\hat{\mathbf{a}}_i^t$ is replaced by a zero vector of the same dimension. The centralised critic retains its standard input $Q_{\phi_i}(x^t, a_1^t, \dots, a_N^t)$. The complete MADDPG+PTAI training loop is given as Algorithm 3 in the appendix.

Interpretation. By conditioning the actor on inferred peer actions, each agent’s policy can explicitly account for the recent joint behaviour of all agents, even without direct communication. In competitive settings this enables agents to develop reactive strategies (e.g. predators coordinating their approach based on each other’s last move) that may be difficult to infer from the current observation alone. Because the AI_Net is pre-trained on random actions—not the current MADDPG policy—it provides a general-purpose, policy-agnostic temporal signal that remains useful throughout training.

4 Environments

We evaluate on four competitive MPE environments from the PettingZoo library [12], all featuring a team of “good” agents (prefixed `agent_`) and a team of adversaries (prefixed

adversary_) that learn concurrently via standard MADDPG co-training. Table 1 summarises the key properties.

Table 1: Competitive MPE environments used in this study.

Environment	Agents	Adversaries	Metric	Description
simple_adversary_v3	2	1	Agent reward	Adversary blends among landmarks to reach its target while good agents try to identify it
simple_push_v3	1	1	Agent win rate	Agent navigates to its goal; adversary tries to block it
simple_tag_v3	1	3	Catches per episode	Three predators cooperate to tag a faster-moving prey
simple_world_comm_v3	2	4	Adversary frames at goal	Adversaries try to reach the good agents' secret target

Physical Deception (simple_adversary_v3). Three landmarks are placed in the arena. One landmark is the adversary’s secret target. The adversary blends in among all landmarks to reach its goal without being identified. The two good agents want to cover all landmarks and determine which the adversary is heading toward. We report the episode reward of the good agent team, averaged over the last 1000 of 30 000 training episodes. Higher values indicate better good-agent performance; the adversary is treated as part of the (co-training) environment from the agents’ perspective.

Cooperative Push (simple_push_v3). A single good agent must navigate to its goal landmark while a single adversary attempts to physically push it away and simultaneously reach its own goal. We measure the *agent win rate*: the fraction of episodes in which the good agent occupies its goal landmark at episode termination.

Predator-Prey (simple_tag_v3). Three predators (adversary team) cooperate to catch a prey (agent team). The prey moves faster than any single predator, requiring the predators to coordinate their pursuit. We measure the mean number of *catches per episode*, where a catch occurs whenever a predator is within tagging distance of the prey. PTAI is particularly relevant here because the predators must anticipate each other’s movements to avoid redundant pursuit directions and successfully encircle the prey.

Keep-Away (simple_world_comm_v3). Four adversaries attempt to reach the true goal landmark that only the good-agent speaker knows. The good agents have a shared communication channel but cannot prevent adversaries from receiving it. We measure *adversary*

frames at goal: the total number of time steps per episode in which at least one adversary occupies the goal landmark. Higher scores represent better adversary performance; PTAI applied to the adversary team benefits from their team-level coordination in locating and converging on the goal.

5 Experiments

5.1 Training Setup

All experiments use a common set of hyperparameters (Table 2) applied to both baseline MADDPG and MADDPG+PTAI.

Table 2: Training hyperparameters (shared across all experiments).

Hyperparameter	Value
Training episodes	30 000
Warmup steps	50 000
Replay buffer capacity	1 000 000
Batch size	1 024
Discount factor γ	0.95
Soft-update rate τ	0.02
Actor/Critic learning rate	0.01
Checkpoint interval	10 000 episodes

Network architecture. All actor networks are two-hidden-layer MLPs (observation $\rightarrow 64 \rightarrow 64 \rightarrow$ action), initialised with Xavier-uniform weights and 0.01 bias constants. Discrete actions are produced via Gumbel-Softmax [4] during training (enabling differentiable action samples) and argmax at evaluation time. Critic networks share the same MLP architecture but take the full joint observation-action vector as input. All gradients are clipped at a global norm of 0.5, and actor losses include a small logit regulariser ($10^{-3} \times \|\text{logits}\|^2$) to prevent policy collapse.

For MADDPG+PTAI, the actor input dimension is expanded by $\sum_k |\mathcal{A}_k|$ to accommodate the inferred previous joint action vector $\hat{\mathbf{a}}_i \in \mathbb{R}^{\sum_k |\mathcal{A}_k|}$.

PTAI pre-training. The AI_Net is pre-trained for 200 episodes of uniformly random play (all agents sample uniformly from their action space) before MADDPG training begins. If a pre-trained `AI_Net.pt` checkpoint already exists in the project root, it is loaded without re-training. The Social and Self-Awareness modules are two-layer MLPs with 64 hidden units, trained with Adam at learning rate 10^{-3} .

5.2 Evaluation Protocol

For each (environment, algorithm) pair we run $n = 3\text{--}4$ independent seeds (see Table 3 for per-cell n). Performance is measured as the mean episode metric over the *last 1 000 training*

episodes of each run, a window that reflects converged policy quality while averaging out short-term volatility. We report the cross-seed mean $\pm$ standard deviation (sample, $n - 1$ denominator). Each independent training seed is treated as the experimental replication; episodes within a run form a dependent learning trajectory and should not be interpreted as thousands of independent replicates. With only 3–4 seeds per condition, the summaries below are therefore descriptive rather than a high-powered inferential comparison.

The metrics reported are:

- **Agent reward** (`simple_adversary_v3`): Episode reward of the good agent team (higher = better for agents).
- **Agent win rate** (`simple_push_v3`): Fraction of episodes in which the good agent is at its goal at episode end (higher = better for agents).
- **Catches per episode** (`simple_tag_v3`): Mean number of predator-prey contact events per episode (higher = better for predators).
- **Adversary frames at goal** (`simple_world_comm_v3`): Mean number of time steps per episode during which at least one adversary occupies the goal landmark (higher = better for adversaries).

For `simple_tag_v3` and `simple_push_v3`, the reported metric is derived from the environment-specific metric tracker rather than raw episode reward. For `simple_adversary_v3`, metric tracking was unavailable for the earliest seed of the baseline (which predates the metric system); we use the raw episode reward directly for all four baseline seeds, which is consistent across runs.

6 Results

6.1 Quantitative Summary

Table 3 reports the main results. Across the observed runs, MADDPG+PTAI has higher mean final-policy performance than baseline MADDPG in all four environments, with relative differences ranging from +83% to +437%.

Table 3: Final-policy performance: mean $\pm$ std over the last 1 000 training episodes, averaged across n independent seeds. Δ is the relative improvement of PTAI over the baseline. All observed mean differences favour PTAI.

Environment	Metric	MADDPG (n)	MADDPG+PTAI (n)	Δ
<code>simple_adversary_v3</code>	Agent reward	9.91 ± 0.40 (4)	20.22 ± 2.82 (4)	+104%
<code>simple_push_v3</code>	Agent win rate	0.18 ± 0.08 (3)	0.33 ± 0.03 (4)	+83%
<code>simple_tag_v3</code>	Catches / ep.	0.62 ± 0.08 (3)	3.31 ± 1.59 (4)	+437%
<code>simple_world_comm_v3</code>	Adv. frames at goal	10.42 ± 1.72 (3)	26.78 ± 2.60 (4)	+157%

Physical Deception. Both algorithms achieve moderate positive agent rewards, but PTAI roughly doubles the baseline’s performance ($9.91 \rightarrow 20.22$). One plausible mechanism is that inferred recent adversary behaviour helps the good agents disambiguate which landmark the adversary is converging on. The larger observed standard deviation for PTAI (± 2.82 vs ± 0.40) motivates further investigation of seed sensitivity with a larger set of independent runs.

Cooperative Push. Agent win rates improve from 0.18 to 0.33 (+83%). The improvement is accompanied by a smaller observed cross-seed standard deviation ($\pm 0.08 \rightarrow \pm 0.03$). With the current number of runs this should not be interpreted as a general stability result, but it motivates a larger-seed analysis of reproducibility. A plausible mechanism is that inferred recent adversary actions help the agent anticipate push directions and adjust its approach to the goal.

Predator-Prey. The most dramatic improvement occurs here: catches per episode rise from 0.62 to 3.31, a 437% increase. This pattern is consistent with the hypothesis that temporal coordination information is especially valuable in a pursuit task. The three predators must spread out and synchronise their movements to corner the faster prey; without an explicit action-history signal, predators can converge on similar pursuit directions, making encirclement less effective. With PTAI, each predator has access to an estimate of where its teammates acted in the previous step and can condition its policy accordingly. The high standard deviation for PTAI (± 1.59) is compatible with substantial seed-to-seed heterogeneity: some runs reach highly effective encirclement behaviours (up to 5.44 catches per episode), while others converge on less effective cooperative behaviours. Establishing whether these represent distinct strategy modes would require a larger number of runs and direct behavioural analysis.

Keep-Away. Adversary frames at goal improve from 10.42 to 26.78 (+157%). A plausible explanation is that inferred recent teammate behaviour helps the adversaries coordinate their approach to the goal. The observed cross-seed standard deviations are ± 1.72 for baseline MADDPG and ± 2.60 for PTAI; with only 3–4 seeds, we avoid drawing a general conclusion about comparative stability from these values alone.

6.2 Training Convergence

Figure 1 shows smoothed training curves (500-episode running average) for both algorithms across all four environments. PTAI reaches higher smoothed performance levels across the observed runs.

Training Convergence: MADDPG vs MADDPG+PTAI
(500-episode smoothing window; shaded = ± 1 std across seeds)

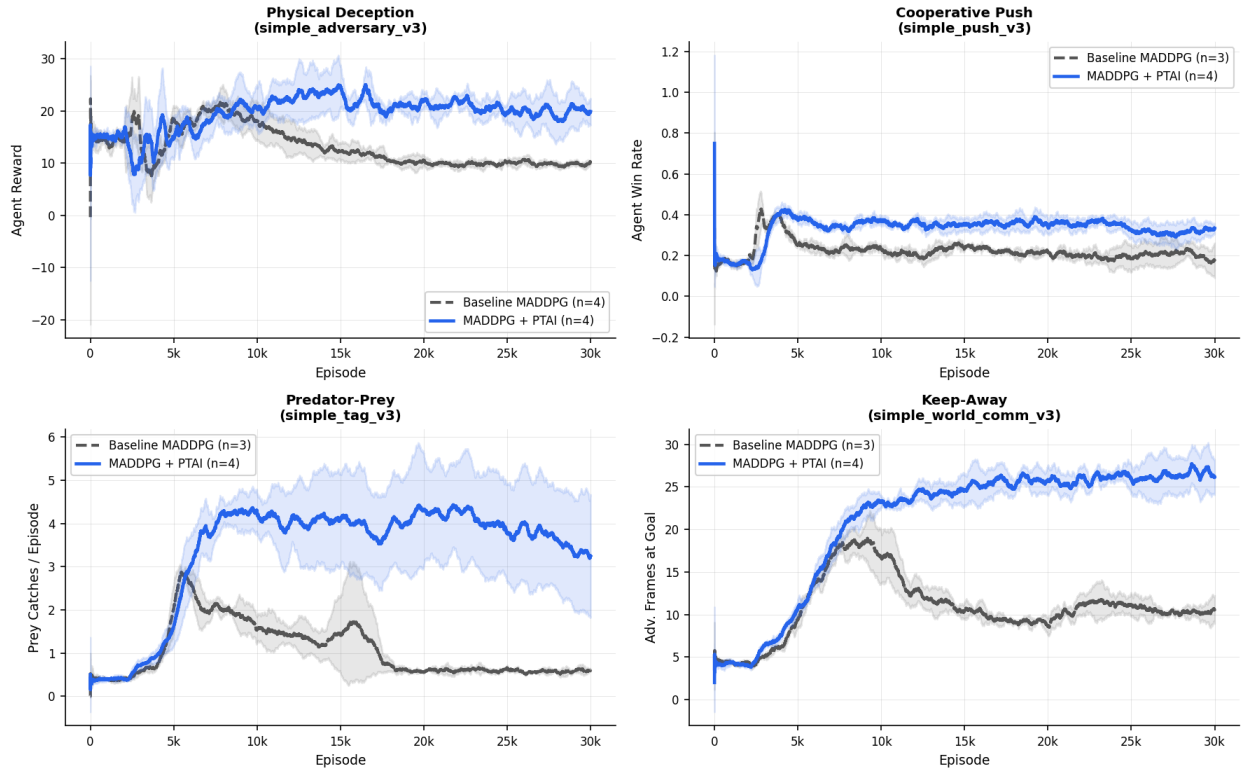


Figure 1: Training convergence across four competitive MPE environments. Lines show the per-episode metric smoothed over a 500-episode window; shaded bands indicate ± 1 standard deviation across seeds. MADDPG+PTAI (blue, solid) shows higher smoothed performance than baseline MADDPG (grey, dashed) across the observed runs in all four environments.

In `simple_adversary_v3` and `simple_push_v3`, PTAI reaches its advantage early in training and maintains it throughout, suggesting that the inferred previous-action signal is useful from the start of learning. In `simple_tag_v3`, both algorithms begin near zero catches and diverge markedly after approximately 10 000 episodes, with PTAI eventually reaching a much higher plateau. In `simple_world_comm_v3`, the gap opens gradually and widens to the end of training, hinting that PTAI’s advantages compound over longer training horizons.

7 Discussion

7.1 Interpretation of the Observed Gains

PTAI addresses a structural information limitation in the MADDPG actor: the actor is conditioned on the current local observation but is not given the previous joint action explicitly. A current observation encodes the consequences of recent interactions, but similar observations may be reached through different action histories. By supplying an estimate of the previous joint action, PTAI augments the actor with a compact summary of how the

current configuration was reached.

This can be viewed as a form of *implicit behavioural inference*. Agents do not need a direct communication channel or access to each other’s policies; instead, AI_Net uses changes in observable quantities to reconstruct recent behaviour. The empirical results are consistent with this mechanism, particularly in Predator-Prey, where coordination between multiple predators is central to successful pursuit. However, the present experiments do not isolate action inference from other explanations, such as the benefit of simply providing the actor with additional temporal information. Establishing the mechanism therefore requires targeted ablations.

7.2 A Statistical View of the Simulation Study

The natural experimental unit in these simulations is the independently initialised training run, not the individual episode. Episodes within a run are serially dependent because each policy update changes the distribution generating subsequent experience. Consequently, the final 1,000-episode average is a summary statistic for one stochastic training trajectory, while uncertainty about algorithm performance is driven primarily by variation across independent seeds.

The current study uses only 3–4 seeds per condition. The reported means and standard deviations are informative descriptive summaries, but they do not support precise estimation of effect sizes or reliable claims about comparative variance. A stronger follow-up design would use a larger, balanced set of matched seeds for MADDPG and PTAI. Using common random seeds where practical would permit paired analysis of the run-level difference

$$D_i = Y_i^{\text{PTAI}} - Y_i^{\text{MADDPG}}, \quad (11)$$

where Y_i is a pre-specified run-level performance measure. Confidence intervals for $\mathbb{E}[D_i]$, together with bootstrap or permutation analyses of paired differences, would quantify uncertainty more directly than relative percentage improvements alone.

Final-policy performance also discards much of the information contained in the learning trajectory. Future analysis should therefore distinguish at least three properties: *learning efficiency*, *final policy quality*, and *run-to-run stability*. Possible run-level summaries include area under the learning curve, episodes required to cross a fixed performance threshold, the final-window mean, and post-convergence variability. These measures would allow a more precise statement of whether PTAI changes how quickly agents learn, where they eventually converge, or both.

7.3 Information-Theoretic Perspective

The motivation for PTAI can also be stated in terms of information about an underlying state. Let S_t denote the latent environment state, O_t^i the local observation of agent i , and A_{t-1} the previous joint action. In a finite discrete setting, conditioning on additional information cannot increase conditional entropy:

$$H(S_t \mid O_t^i, A_{t-1}) \leq H(S_t \mid O_t^i). \quad (12)$$

The reduction is

$$I(S_t; A_{t-1} \mid O_t^i) = H(S_t \mid O_t^i) - H(S_t \mid O_t^i, A_{t-1}), \quad (13)$$

which is positive precisely when the previous joint action contains state information not already captured by the current local observation. This does not by itself imply improved control performance, and PTAI observes only an estimated action vector rather than the true A_{t-1} . It nevertheless provides a formal version of the intuition behind the method: action-history information can reduce uncertainty when the current observation is not sufficient to reconstruct the relevant recent interaction history.

7.4 Limitations and Planned Evaluation

Several limitations of the present study point directly to the next experiments. First, the observation decomposition used by PTAI is hand-specified for each environment, so the current architecture assumes that features can be routed to the appropriate Directional modules. Attention-based or graph-based entity representations could remove this dependency and support variable numbers of agents.

Second, the study does not yet report AI_Net action-prediction accuracy as a separate quantity. A direct evaluation should measure prediction accuracy or probabilistic scoring metrics by agent and environment, then relate inference quality to downstream policy performance. Varying the amount of pre-training data would help determine whether there is a threshold beyond which more accurate action inference provides little additional control benefit.

Third, an ablation study is needed to isolate the source of PTAI’s gains. Useful comparisons include standard MADDPG, MADDPG given two consecutive observations directly, MADDPG given only the agent’s own previous action, PTAI with inferred joint actions, and an oracle condition given the true previous joint action. The oracle condition would provide an upper reference for the value of action-history information, while the raw history condition would test whether explicit action inference contributes beyond simply increasing temporal context.

Fourth, robustness to inference error can be studied directly by corrupting an oracle or PTAI action signal with controlled probability and estimating downstream performance as a function of the corruption rate. This would identify how accurate behavioural inference must be before it becomes useful and whether incorrect inferred actions eventually become harmful.

Finally, the partial-observability interpretation should be tested rather than assumed. Controlled observation noise, feature masking, or restricted sensing could be used to vary how much information is available in O_t^i . If PTAI’s advantage increases as observability decreases, that would provide stronger evidence for the proposed mechanism. Other extensions include longer temporal histories, recurrent or attention-based action inference, online refinement of AI_Net, continuous action spaces, larger agent populations, and fully cooperative environments.

8 Conclusion

We presented PTAI, a pre-trained action inference module that augments each MADDPG actor with estimates of the previous joint action derived from temporally stacked observations. Across the four competitive MPE environments studied here, MADDPG+PTAI achieves higher mean final-policy performance than baseline MADDPG in every environment, with observed relative improvements ranging from +83% to +437%. The largest observed gain occurs in Predator-Prey, where successful behaviour requires coordination among multiple predators, which is consistent with the hypothesis that recent joint-action information is especially useful in coordination-intensive settings.

The frozen AI_Net avoids introducing an additional online optimisation objective into MADDPG, while adding only forward-pass inference to construct the augmented actor input. The present results establish PTAI as a promising empirical extension, but the small number of independent seeds and absence of mechanism-focused ablations limit the strength of statistical and causal conclusions that can be drawn from the current study.

The most important next step is therefore not simply to add more environments, but to test *why* and *when* action inference helps. Planned directions include larger paired-seed simulation studies with confidence intervals; analysis of complete learning trajectories rather than only final-window averages; direct measurement of action-inference quality; ablations against raw observation history, self-action history, and oracle joint actions; and controlled corruption experiments linking inference error to downstream performance. Testing PTAI under systematically reduced observability would further assess whether it acts as a general mechanism for recovering useful temporal information in decentralised multi-agent decision-making. Architectural extensions based on attention or graph representations, together with continuous-action and cooperative settings, offer a path toward evaluating the approach beyond the hand-structured MPE environments used here.

Acknowledgments

We would like to express our sincere gratitude to Professor Tao Gao for his invaluable guidance throughout this project. Our work began with a study of the foundational principles of reinforcement learning as presented in the textbook by Sutton and Barto [10]. Following Professor Gao’s recommendation, we explored various implementations of deep reinforcement learning, drawing on the works of Yu et al. [13] and Chen et al. [2], which led us to the domain of multi-agent reinforcement learning and ultimately to the MADDPG algorithm of Lowe et al. [5], later made available via the PettingZoo library by the Farama Foundation [12]. We integrated the MADDPG algorithm by adapting an existing codebase from Git-123-Hub [3], modifying it substantially to suit our project. Our code is available at <https://github.com/cron08/maddpg-proj>.

References

- [1] L. Busoniu, R. Babuška, and B. De Schutter. A comprehensive survey of multi-agent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C*,

- 38(2):156–172, 2008.
- [2] Y. F. Chen, M. Everett, M. Liu, and J. P. How. Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning, 2016.
 - [3] Git-123-Hub. *maddpg-pettingzoo-pytorch*. <https://github.com/Git-123-Hub/maddpg-pettingzoo-pytorch>, 2021. GitHub repository.
 - [4] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-Softmax. In *ICLR*, 2017.
 - [5] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *NeurIPS*, 2017.
 - [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller. Playing Atari with deep reinforcement learning. In *NeurIPS*, 2013.
 - [7] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Harley, T. P. Lillicrap, D. Silver, and K. Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *ICML*, 2016.
 - [8] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *ICML*, 2014.
 - [9] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. In *NeurIPS*, 1999.
 - [10] R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2018.
 - [11] M. Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proceedings of ICML*, pages 330–337, 1993.
 - [12] J. Terry, B. Black, N. Grammel, M. Jayakumar, A. Hari, R. Sullivan, L. S. Santos, C. Diallo, C. Horsch, C. Schwartz, M. Veale, and B. Zolfaghari. PettingZoo: Gym for multi-agent reinforcement learning. In *NeurIPS*, 2021.
 - [13] C. Yu, A. Velu, E. Vinitzky, Y. Wang, A. Bayen, and Y. Wu. The surprising effectiveness of MAPPO in cooperative, multi-agent games, 2021.

A Algorithms

Algorithm 1 Multi-Agent Deep Deterministic Policy Gradient (MADDPG) for N agents

```

1: for episode = 1 to  $M$  do
2:   Receive initial observation  $x$  and local observations  $o_1, \dots, o_N$ 
3:   for  $t = 1$  to  $T$  do
4:     for each agent  $i$  do
5:       Select action  $a_i = \mu_{\theta_i}(o_i) + \mathcal{N}_t$  (exploration noise)
6:     end for
7:     Execute joint action  $a = (a_1, \dots, a_N)$ , observe reward  $r$  and next state  $x'$ 
8:     Store  $(x, a, r, x')$  in replay buffer  $\mathcal{D}$ 
9:      $x \leftarrow x'$ 
10:    for each agent  $i$  do
11:      Sample minibatch of  $S$  transitions  $(x^j, a^j, r^j, x'^j)$  from  $\mathcal{D}$ 
12:      Set  $y^j = r^j + \gamma Q_{\phi'_i}(x'^j, a'^j_1, \dots, a'^j_N)$  where  $a'^j_k = \mu'_k(o^j_k)$ 
13:      Update critic by minimising  $\mathcal{L}(\phi_i) = \frac{1}{S} \sum_j (y^j - Q_{\phi_i}(x^j, a^j_1, \dots, a^j_N))^2$ 
14:      Update actor using the sampled policy gradient:  $\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o^j_i) \nabla_{a_i} Q_{\phi_i}(x^j, a^j_1, \dots, a^j_N) \big|_{a_i = \mu_i(o^j_i)}$ 
15:    end for
16:    for each agent  $i$  do
17:      Update target networks:  $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ ,  $\phi'_i \leftarrow \tau \phi_i + (1 - \tau) \phi'_i$ 
18:    end for
19:  end for
20: end for

```

Algorithm 2 Pre-Training the AI_Net (PTAI) on random episodes

```
1: for episode = 1 to  $P$  do
2:   Receive initial state  $\mathbf{x}$  and observation  $o$ 
3:   for  $t = 1$  to  $T$  do
4:     for each agent  $i$  do
5:        $a_i \leftarrow \mathcal{A}_i.\text{sample}()$  (uniform random action)
6:     end for
7:      $\mathbf{o}^- \leftarrow \mathbf{o}$ 
8:     Execute actions  $a = (a_1, \dots, a_N)$ , observe reward  $r$  and new observations  $o$ 
9:     for each agent  $i$  (observer) do
10:       $\Delta o_i \leftarrow o_i - o_i^-$ 
11:      for each agent  $k$  (observed) do
12:        if  $k = i$  then
13:          Update Self-Awareness module:  $\mathcal{L}(\theta) = \text{MSE}(\text{onehot}(a_{i,i}) -$   

Self $_{\theta}(o_{i,i}, o_{i,g}, o_{i,i}^-, o_{i,g}^-, \Delta o_{i,i}, \Delta o_{i,g}))$ 
14:        else
15:          Update Social-Awareness module:  $\mathcal{L}(\theta) = \text{MSE}(\text{onehot}(a_{i,k}) -$   

Social $_{\theta}(o_{i,k}, o_{i,i}, o_{i,g}, o_{i,k}^-, o_{i,i}^-, o_{i,g}^-, \Delta o_{i,k}, \Delta o_{i,i}, \Delta o_{i,g}))$ 
16:        end if
17:      end for
18:    end for
19:  end for
20: end for
21: Freeze all AI_Net parameters.
```

Algorithm 3 MADDPG with Pre-Trained Action Inference (MADDPG+PTAI) for N agents

```

1: Pre-train AI_Net (Algorithm 2) and freeze weights.
2: for episode = 1 to  $M$  do
3:   Receive initial observations  $o_1^1, \dots, o_N^1$ 
4:   for each agent  $i$  do
5:      $a_i^1 \leftarrow \mu_{\theta_i}(o_i^1, \mathbf{0}) + \mathcal{N}_t$  ( $\hat{\mathbf{a}}$  is zero at first step)
6:   end for
7:    $\mathbf{o}^- \leftarrow \mathbf{o}$ 
8:   for  $t = 2$  to  $T$  do
9:     for each agent  $i$  do
10:       $\hat{\mathbf{a}}_i^t = \text{AI\_Net}(o_i^t, o_i^{t-1})$  (infer previous joint action from observation delta)
11:       $a_i^t \leftarrow \mu_{\theta_i}(o_i^t, \hat{\mathbf{a}}_i^t) + \mathcal{N}_t$ 
12:    end for
13:    Execute  $a = (a_1^t, \dots, a_N^t)$ , observe reward  $r$ , next observations  $o^{t+1}$ 
14:    Store  $(x^t, a^t, r^t, x^{t+1}, o^{t+1})$  in replay buffer  $\mathcal{D}$ 
15:     $\mathbf{o}^- \leftarrow \mathbf{o}$ 
16:    for agent  $i = 1$  to  $N$  do
17:      Sample minibatch of  $S$  tuples  $(x^j, a^j, r^j, x'^j, o^{-j})$  from  $\mathcal{D}$ 
18:      Compute  $\hat{\mathbf{a}}_i^{j+1} = \text{AI\_Net}(o_i^{j+1}, o_i^j)$  for target
19:       $y_i^j = r_i^j + \gamma Q_{\phi_i'}(x'^j, a_1'^j, \dots, a_N'^j)$  where  $a_k'^j = \mu'_k(o_k^{j+1}, \hat{\mathbf{a}}_k^{j+1})$ 
20:      Update critic:  $\mathcal{L}(\phi_i) = \frac{1}{S} \sum_j (y_i^j - Q_{\phi_i}(x^j, a_1^j, \dots, a_N^j))^2$ 
21:       $\hat{\mathbf{a}}_i^j = \text{AI\_Net}(o_i^j, o_i^{-j})$ 
22:      Update actor gradient:  $\nabla_{\theta_i} J \approx \frac{1}{S} \sum_j \nabla_{\theta_i} \mu_i(o_i^j, \hat{\mathbf{a}}_i^j) \nabla_{a_i} Q_{\phi_i}(x^j, a_1^j, \dots, a_i, \dots, a_N^j) \Big|_{a_i = \mu_i(o_i^j, \hat{\mathbf{a}}_i^j)}$ 
23:    end for
24:    for each agent  $i$  do
25:       $\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$ 
26:    end for
27:  end for
28: end for

```
